

1. gRPC E2E テスト手順書

Go 製の図書照会・計算配信サービスに対して、runnora で Unary RPC と Server streaming RPC を実行する。API 定義は `proto/library.proto`、runbook は `runbooks/` に置く。

`grpc-test/` で `./scripts/run.ps1` を実行してテストし、`./scripts/build-docs.ps1` でこの手順書を生成する。前処理用 DB、oapi2wire、モックサーバは使用しない。

2. Unary RPC

GetBook で B0003 を照会する。入力と期待する応答全体は `cases/unary/` の JSON ファイルから読み込む。

表 2-1 テストシナリオ

シナリオ	Unary RPC で蔵書を 1 冊取得する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	B0003 のタイトル、ジャンル、在庫数を確認する Unary sample.library.v1.LibraryService/GetBook	表 2-3 get-book-request.json	期待値 : [gRPC vars.expected.status] 表 2-4 get-book-expected.json	

表 2-2 gRPC 呼び出し情報 (Unary RPC で蔵書を 1 冊取得する)

手順	Runner	Address	Service/Method	RPC type	Metadata	Timeout
1	greq	127.0.0.1:19090	sample.library.v1.LibraryService/ GetBook	Unary		

表 2-3 gRPC 送信メッセージ (Unary RPC で蔵書を 1 冊取得する / 手順 1)

出典	JSON path	値
参照式	\$	"{{ vars.request }}"
get-book-request.json	\$.book_id	B0003

表 2-4 期待値・検証条件 (Unary RPC で蔵書を 1 冊取得する / 手順 1) (1/2)

区分	出典	内容
検証式	unary.yml:18	current.res.status == vars.expected.status && compare(current.res.message, vars.expected.message)
参照 JSON	get-book-expected.json	\$.message.book.available_copies: 2
		\$.message.book.book_id: B0003

表 2-4 期待値・検証条件（Unary RPC で蔵書を 1 冊取得する／手順 1）（2／2）

区分	出典	内容
		\$.message.book.genre: TECH
		\$.message.book.title: Go 言語による Web アプリ開発
		\$.status: 0

3. Server streaming RPC

Unary `GetBook` の応答からジャンルを `bind` で保存し、JSON ファイルで指定した `ListBooks` の検索条件と照合する。受信したメッセージ配列を期待 JSON と比較し、件数、順序、内容を検証する。

表 3-1 テストシナリオ

シナリオ	Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	検索条件に使う蔵書のジャンルを Unary RPC で取得する Unary <code>sample.library.v1.LibraryService/GetBook</code>	表 3-3 <code>get-book-request.json</code>	期待値 : [gRPC <code>vars.get_expected.status</code>] 表 3-5 <code>get-book-expected.json</code>	
2	Unary 応答のジャンルを保存し、JSON ファイルの検索条件と 照合する 変数を設定			
3	TECH の蔵書が ID 順に 2 件流れることを確認する Server streaming <code>sample.library.v1.LibraryService/ListBooks</code>	表 3-4 <code>list-books-request.json</code>	期待値 : [gRPC <code>vars.list_expected.status</code>] 表 3-6 <code>list-books-expected.json</code> <code>list-books-request.json</code>	

表 3-2 gRPC 呼び出し情報 (Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する)

手順	Runner	Address	Service/Method	RPC type	Metadata	Timeout
1	greq	127.0.0.1:19090	<code>sample.library.v1.LibraryService/GetBook</code>	Unary		
3	greq	127.0.0.1:19090	<code>sample.library.v1.LibraryService/ListBooks</code>	Server streaming		

表 3-3 gRPC 送信メッセージ（Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する／手順 1）

出典	JSON path	値
参照式	\$	"{{ vars.get_request }}"
get-book-request.json	\$.book_id	B0003

表 3-4 gRPC 送信メッセージ（Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する／手順 3）

出典	JSON path	値
参照式	\$	"{{ vars.list_request }}"
list-books-request.json	\$.genre	TECH

表 3-5 期待値・検証条件（Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する／手順 1）

区分	出典	内容
検証式	server-streaming.yml:20	current.res.status == vars.get_expected.status && compare(current.res.message, vars.get_expected.message)
参照 JSON	get-book-expected.json	\$.message.book.available_copies: 2
		\$.message.book.book_id: B0003
		\$.message.book.genre: TECH
		\$.message.book.title: Go 言語による Web アプリ開発
		\$.status: 0

表 3-6 期待値・検証条件（Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する／手順 3）（1／2）

区分	出典	内容
検証式	server-streaming.yml:32	current.res.status == vars.list_expected.status && vars.list_request.genre == genre && compare(current.res.messages, vars.list_expected.messages)
参照 JSON	list-books-expected.json	\$.messages[0].available_copies: 2
		\$.messages[0].book_id: B0003
		\$.messages[0].genre: TECH

表 3-6 期待値・検証条件（Unary の応答と検索条件の整合を確認して Server streaming RPC を検証する／手順 3）（2／2）

区分	出典	内容
		\$.messages[0].title: Go 言語による Web アプリ開発
		\$.messages[1].available_copies: 1
		\$.messages[1].book_id: B0004
		\$.messages[1].genre: TECH
		\$.messages[1].title: データベース入門
		\$.status: 0
	list-books-request.json	\$.genre: TECH

4. 最終回だけに集計結果を含む配信

Calculate は入力値の累計を 1 件ずつ送る。最後の更新だけに optional completion を設定し、件数と合計を送る。入力と期待するメッセージ配列は cases/calculation-streaming/ の JSON ファイルで管理する。期待 JSON では途中 2 件に completion を置かず、最後の 1 件だけに置く。配列全体の比較で有無、順序、値を確認する。

表 4-1 テストシナリオ

シナリオ	計算結果を逐次配信し、最後のメッセージだけに集計結果を含める			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	各回の累計を受け取り、最終回だけ completion があることを 確認する Server streaming sample.library.v1.LibraryService/Calculate	表 4-3 request.json	期待値 : [gRPC vars.expected.status] 表 4-4 expected.json	

表 4-2 gRPC 呼び出し情報（計算結果を逐次配信し、最後のメッセージだけに集計結果を含める）

手順	Runner	Address	Service/Method	RPC type	Metadata	Timeout
1	greq	127.0.0.1:19090	sample.library.v1.LibraryService/ Calculate	Server streaming		

表 4-3 gRPC 送信メッセージ（計算結果を逐次配信し、最後のメッセージだけに集計結果を含める／手順 1）

出典	JSON path	値
参照式	\$	"{{ vars.request }}"
request.json	\$.values[0]	2
request.json	\$.values[1]	3
request.json	\$.values[2]	5

表 4-4 期待値・検証条件（計算結果を逐次配信し、最後のメッセージだけに集計結果を含める／手順 1）

区分	出典	内容
検証式	calculation-streaming .yaml:19	current.res.status == vars.expected.status && current.res.messages[0].completion == nil && current.res.messages[1].completion == nil && current.res.messages[2].completion != nil && compare(current.res.messages, vars.expected.messages)
参照 JSON	expected.json	\$.messages[0].running_total: 2
		\$.messages[0].step: 1
		\$.messages[1].running_total: 5
		\$.messages[1].step: 2
		\$.messages[2].completion.count: 3
		\$.messages[2].completion.total: 10
		\$.messages[2].running_total: 10
		\$.messages[2].step: 3
		\$.status: 0